

个人答辩报告

12312505 江博成

SUSTC

May 14, 2025

Content

① 负责工作简述

② Controller

负责工作简述

- ① 顶层架构与接口设计
- ② Controller 控制信号生成模块
- ③ Decoder 立即数生成与寄存器模块
- ④ WriteBackMUX 写回 MUX 模块
- ⑤ 基本测试场景 1,2

Controller

```
1 module Controller (
2     input  [31:0] inst,
3     output reg  branch,
4     output reg  [1:0] alu_op,
5     output reg  alu_src,
6     output reg  mem_read,
7     output reg  mem_write,
8     output reg  mem_to_reg,
9     output reg  reg_write,
10    output reg  is_jal,
11    output reg  is_jalr,
12    output reg  is_lui,
13    output reg  is_auiipc,
14    output reg  is_ecall,
15    output reg  en_pc,
16    output reg  en_input,
17    output reg  en_output,
18    input  [31:0] reg_a7, // ecall
19    need to check
20    input  done_input // let
21    en_pc = 1
22);
```

```
1 // Opcode definitions (parameterized
2     )
3     parameter R_TYPE  = 7'b0110011;
4     parameter I_TYPE1 = 7'b0010011;
5         // addi
6     parameter I_TYPE2 = 7'b0000011;
7         // lw
8     parameter I_TYPE3 = 7'b1100111;
9         // jalr
10    parameter I_TYPE4 = 7'b1110011;
11        // ecall
12    parameter S_TYPE  = 7'b0100011;
13    parameter B_TYPE  = 7'b1100011;
14    parameter U_TYPE1 = 7'b0110111;
15        // lui
16    parameter U_TYPE2 = 7'b0010111;
17        // auipc
18    parameter J_TYPE  = 7'b1101111;
19        // jal
20    wire [6:0] opcode = inst[6:0];
```

Controller

```
1 always @(*) begin
2     // Default all outputs
3     branch      = 0;
4     alu_op       = 2'b11;
5     alu_src      = 0;
6     mem_read     = 0;
7     mem_write    = 0;
8     mem_to_reg   = 0;
9     reg_write    = 0;
10    is_jal        = 0;
11    is_jalr       = 0;
12    is_lui        = 0;
13    is_auiopc     = 0;
14    is_ecall      = 0;
15    en_input      = 0;
16    en_output     = 0;
17    en_pc         = 1;
18    case (opcode)
19        R_TYPE: begin
20            alu_op      = 2'b10;
21            alu_src     = 0;
22            reg_write   = 1;
23        end
```

```
1 I_TYPE1: begin
2     alu_op       = 2'b11; // check here
3     alu_src      = 1;
4     reg_write    = 1;
5 end
6 I_TYPE2: begin // lw
7     alu_op       = 2'b00;
8     alu_src      = 1;
9     mem_read     = 1;
10    mem_to_reg    = 1;
11    reg_write     = 1;
12 end
13 I_TYPE3: begin // jalr
14     alu_op       = 2'b11;
15     alu_src      = 1;
16     reg_write    = 1;
17     is_jalr      = 1;
18 end
19 S_TYPE: begin // sw
20     alu_op       = 2'b00;
21     alu_src      = 1;
22     mem_write    = 1;
23 end
```

Controller

```
1 I_TYPE4: begin // ecall
2     is_ecall    = 1;
3     case (reg_a7)
4         1, 34, 35: begin
5             en_output=1;
6         end
7         5: begin
8             reg_write=1;
9             en_input=1;
10            if (done_input) en_pc=1;
11            else en_pc=0;
12        end
13        default begin
14
15        end
16    endcase
17 end
18 B_TYPE: begin // beq, bne, etc.
19     alu_op      = 2'b01;
20     branch      = 1;
21 end
```

```
1 U_TYPE1: begin // lui
2     alu_op      = 2'b11;
3     alu_src      = 1;
4     reg_write    = 1;
5     is_lui       = 1;
6 end
7 U_TYPE2: begin // auipc
8     alu_op      = 2'b11;
9     alu_src      = 1;
10    reg_write    = 1;
11    is_auipc     = 1;
12 end
13 J_TYPE: begin // jal
14     alu_op      = 2'b11; // don't
15                    care
16     alu_src      = 1; // don't care
17     reg_write    = 1;
18     is_jal       = 1;
19 end
20 default: begin
21     en_pc        = 0;
22     // All control signals remain
23     default
24 end
```

Testbench of Controller

```
1 module tb_Controller;
2
3   // inputs
4   reg [31:0] inst;
5   reg [31:0] reg_a7;
6   reg        done_input;
7
8   // outputs
9   wire        branch;
10  wire [1:0]   alu_op;
11  wire        alu_src;
12  wire        mem_read;
13  wire        mem_write;
14  wire        mem_to_reg;
15  wire        reg_write;
16  wire        is_jal;
17  wire        is_jalr;
18  wire        is_lui;
19  wire        is_auipc;
20  wire        is_ecall;
21  wire        en_pc;
22  wire        en_input;
23  wire        en_output;
```

```
1   Controller uut (
2     .inst      (inst),
3     .branch    (branch),
4     .alu_op    (alu_op),
5     .alu_src   (alu_src),
6     .mem_read  (mem_read),
7     .mem_write (mem_write),
8     .mem_to_reg(mem_to_reg),
9     .reg_write (reg_write),
10    .is_jal    (is_jal),
11    .is_jalr   (is_jalr),
12    .is_lui    (is_lui),
13    .is_auipc  (is_auipc),
14    .is_ecall  (is_ecall),
15    .en_pc     (en_pc),
16    .en_input  (en_input),
17    .en_output (en_output),
18    .reg_a7    (reg_a7),
19    .done_input(done_input)
20  );
21  initial begin
```

Testbench of Controller

```
1 // dump waveform
2 $dumpfile("tb_Controller.vcd");
3 $dumpvars(0, tb_Controller);
4 // ----- Test R_TYPE -----
5 inst = {25'd0, 7'b0110011}; //
   R_TYPE opcode
6 #1;
7 if (alu_op != 2'b10 || reg_write
    != 1) $fatal("R_TYPE_failed");
8 // ----- Test ADDI (I_TYPE1) -----
9 inst = {20'd0, 5'd1, 3'b000, 5'd2,
    7'b0010011};
10 #1;
11 if (alu_src != 1 || alu_op != 2'
    b11 || reg_write != 1) $fatal(
    "ADDI_failed");
12 // ----- Test LW (I_TYPE2) -----
13 inst = {20'd0, 5'd3, 3'b010, 5'd4,
    7'b0000011};
14 #1;
15 if (!(mem_read && mem_to_reg &&
    reg_write && alu_src==1 &&
    alu_op==2'b00))
16   $fatal("LW_failed");
```

```
1 // ----- Test ECALL input done -----
2 done_input = 1;
3 #1;
4 if (en_pc!=1) $fatal("ECALL_input_
    done_failed");
5 // ----- Test SW (S_TYPE) -----
6 inst = {20'd0, 5'd7, 5'd8, 3'b010,
    5'd0, 7'b0100011};
7 #1;
8 if (!(mem_write && alu_src && alu_op
    ==2'b00)) $fatal("SW_failed");
9 // ----- Test BEQ (B_TYPE) -----
10 inst = {7'b00000000, 5'd0, 5'd0, 3'
    b000, 4'b0000, 1'b0, 7'b1100011
    };
11 #1;
12 if (!(branch && alu_op==2'b01))
    $fatal("BEQ_failed");
13 // ----- Test LUI (U_TYPE1) -----
14 inst = {20'hABCD0, 5'd0, 7'b0110111
    };
15 #1;
16 if (!(is_lui && reg_write && alu_src
    )) $fatal("LUI_failed");
```


Testbench of Controller

```
1 // ----- Test AUIPC (U_TYPE2)
  -----
2 inst = {20'h10000, 5'd0, 7'
    b00101111};
3 #1;
4 if (!(is_auipc && reg_write &&
    alu_src)) $fatal("AUIPC_
    failed");
5 // ----- Test JAL (J_TYPE) -----
6 inst = {20'd0,5'd0,3'b000,5'd0
    ,7'b1101111};
7 #1;
8 if (!(is_jal && reg_write))
    $fatal("JAL_failed");
9 $display("All_Controller_tests_
    passed");
10 #1; $finish;
11 end
12
13 endmodule
```

1 // 说明: Controller模块主要接受的input的inst中的opcode, testbench生成了具有不同opcode的inst作为Controller模块的输入。

2 // 对于课上没有提到如何实现的指令, 我们直接增加了is_开头的控制信号, 来做额外的分类讨论。

3 // 对于输入输出, 我们是直接实现的ecall, 也就需要额外的输入信号reg_a7。

4 // 若reg_a7为1,34,35, 则对应输出, 令使能信号en_output=1, output模块接受该信号执行输出判断。

5 // 若reg_a7为5, 则对应输入, 因为输入需要等待确认信号, 不能在一个时钟周期内完成, 故令使能信号en_pc=0, 各模块接受该信号中断时序, 并令en_input=1, 传给input模块并参与后续逻辑, 等待input模块传回done_input型号, 令en_pc=1重新启动时序。

6 // 以上逻辑测试在testbench代码中均有显现, 波形图也符合设想逻辑。

7 // 最终Tcl console输出为: All Controller tests passed

Waveform of Controller



Figure: Waveform of Controller

THANK YOU!